University Mohammed VI Polytechnic

College of Computing

Java Programming & Software Development

Prof. Fahd Kalloubi
Fahd.kalloubi@um6p.ma

- Associate professor in Machine Learning and Big Data
- Ex Adjunct professor at euro-Mediterranean university of Fez
- Affiliate professor at UM6P – College of Computing
- Ex Professor at ENSIAS
- Research interests: Machine learning, Graph Neural Networks, NLP, Deep Learning, Big Data

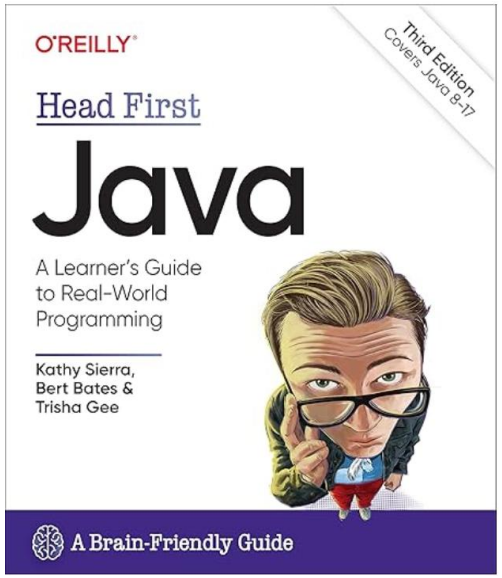


Lecture Topics

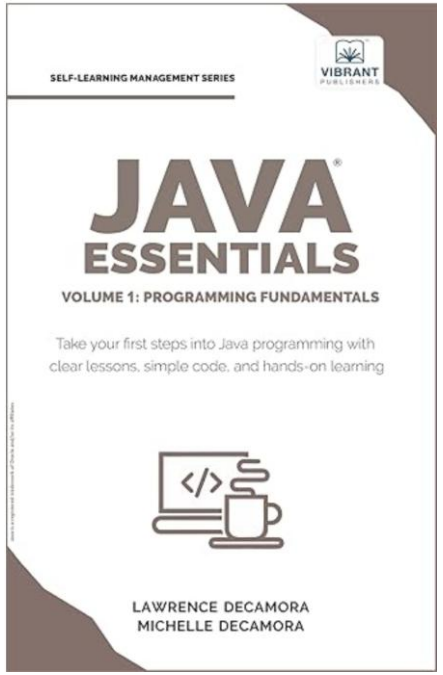
1. Java IO Streams
2. Java collections: Enumeration
3. Generalization and Abstraction
4. Abstract method vs Concrete method
5. Abstract class
6. Interface vs Abstract class
7. Interface enhancements from JDK 8 and JDK 9

Literature

Head First Java: A Brain-Friendly Guide, 3rd Edition
Kathy Sierra, Bert Bates, Trisha Gee
O'REILLY
June 21, 2022



Java Essentials, Volume 1: Programming Fundamentals: Learn Java, Variables, Control Flow, and Methods with Hands-On Coding Exercises and Examples for Beginners
Dr. Lawrence G. Decamora III, Michelle C. Decamora
Vibrant Publishers
September 25, 2025



This Lecture



Instructional Objectives

After completing this part, you should be able to:

- Understand Java IO streams (byte vs character, buffered streams)
- Understand Enumeration
- Understand Abstraction concept
- Difference between generalization and Abstraction
- Difference between Interface and Abstract class
- Understand Interface enhancements in JDK
- Understand private methods and default methods in Interfaces



Learning outcomes

At the end part, you are expected to:

- Read and write files using Java IO streams
- Implement java programs using Enumeration collection
- Understand when to use Interface and Abstract class
- Implement default and static methods in Interface

Java IO Streams: what is a stream?

- A **stream** is an ordered sequence of data flowing from a **source** to a **destination**, one piece at a time.
- Java programs **read** from an input stream and **write** to an output stream.
- The source or destination can be a file, the keyboard, the console, a network socket, or even an array in memory.
- The classes live in the package `java.io` (plus the newer `java.nio.file` helpers).



Byte Streams vs Character Streams

Byte streams

abstract roots: InputStream / OutputStream

- Read and write raw **8-bit bytes**.
- Use for **binary data**: images, audio, video, .class files, any file "as is".
- `read()` returns an `int` from 0 to 255, or **-1** at the end of the stream.
- Class names end in **Stream**: `FileInputStream`, `BufferedOutputStream`...

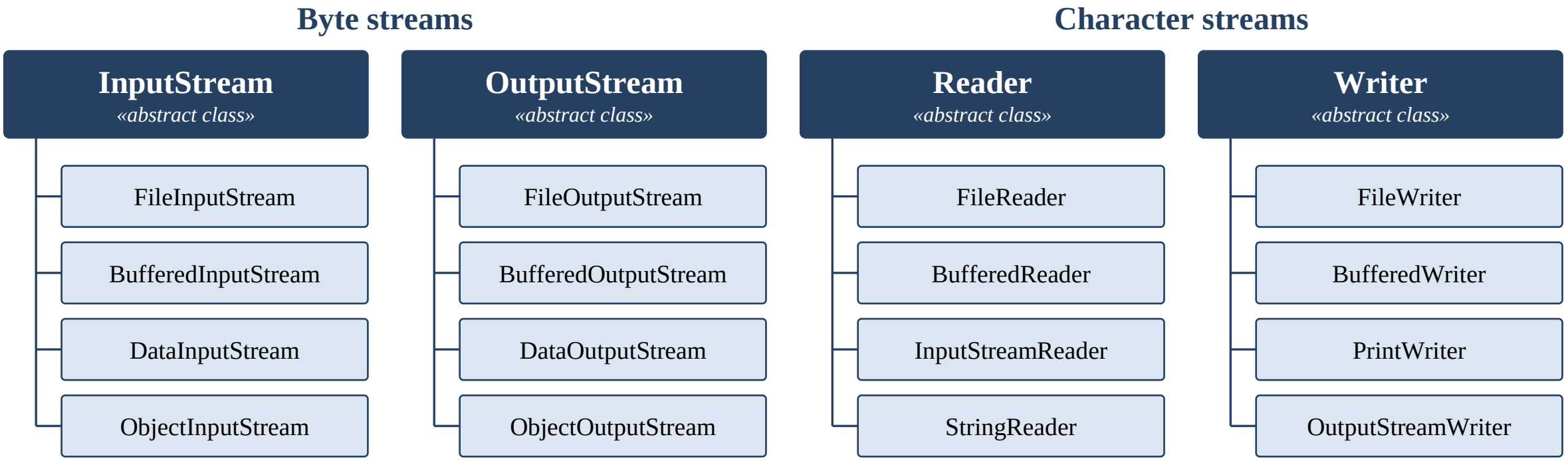
Character streams

abstract roots: Reader / Writer

- Read and write **16-bit Unicode characters**.
- Use for **text**: .txt, .csv, .java, .json, .html files.
- Handle the character encoding (UTF-8, ...) for you.
- Class names end in **Reader / Writer**: `FileReader`, `BufferedWriter`...

Rule of thumb: **text** → `Reader` / `Writer`; **everything else** → `InputStream` / `OutputStream`.

The java.io Class Hierarchy



- All four roots are **abstract classes**. For example, `InputStream` declares `public abstract int read()`.
- Each concrete subclass implements `read()` / `write()` for its own source: a file, a buffer, another stream...
- Your code can use the abstract type (`InputStream in = ...`) and not care where the bytes come from: **abstraction in action**

Closing Streams: try-with-resources

Before Java 7: close manually in finally

```
FileReader r = null;
try {
    r = new FileReader("a.txt");
    // ... use r
} catch (IOException e) {
    e.printStackTrace();
} finally {
    if (r != null) {
        try { r.close(); }
        catch (IOException e) { }
    }
}
```

Java 7+: try-with-resources

```
try (FileReader r = new FileReader("a.txt")) {
    // ... use r
} catch (IOException e) {
    e.printStackTrace();
}
// r.close() is called automatically,
// even if an exception is thrown
```

- A stream holds an operating-system resource (a file handle): it must **always be closed**.
- Any object that implements the `AutoCloseable` **interface** can be declared in `try ( ... )`; several resources are separated by `;`.
- Most IO methods throw the **checked** exception `IOException` (e.g. `FileNotFoundException`): catch it or declare `throws IOException`.

Example 1: Copying a File with Byte Streams

- `FileInputStream` reads bytes from a file; `FileOutputStream` writes bytes to a file.
- `read()` returns the next byte, or **-1** when the end of the file is reached.
- Works for **any** file type, because it never interprets the bytes.
- Both streams are closed automatically at the end of the `try` block.

```
import java.io.*;

public class CopyFile {
    public static void main(String[] args) {
        try (FileInputStream in = new FileInputStream("photo.jpg");
             FileOutputStream out = new FileOutputStream("copy.jpg")) {
            int b;
            while ((b = in.read()) != -1) {   // -1 = end of stream
                out.write(b);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Example 2: Text Files with FileWriter and FileReader

```java
// 1) Write text to a file
try (FileWriter writer = new FileWriter("notes.txt")) {
    writer.write("Hello, Java IO!\n");
    writer.write("Streams are fun.");
} catch (IOException e) {
    e.printStackTrace();
}

// 2) Read it back, one character at a time
try (FileReader reader = new FileReader("notes.txt")) {
    int ch;
    while ((ch = reader.read()) != -1) {
        System.out.print((char) ch);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

- `FileWriter` creates the file (or **overwrites** it). Use `new FileWriter("notes.txt", true)` to **append**.
- `FileReader.read()` returns one character as an `int`: cast it with `(char)`.
- Reading one character at a time works, but it is **slow** for large files...

Output:

```
Hello, Java IO!
Streams are fun.
```

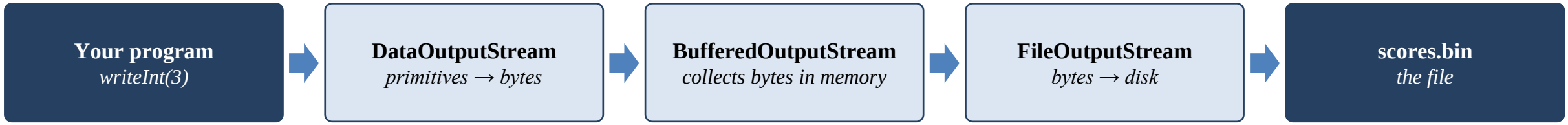
Example 3: Buffered Streams (line by line)

- A **buffer** reads/writes a big block of data at once and keeps it in memory, so far fewer slow disk accesses are needed.
- `BufferedReader.readLine()` returns a whole line (without the line break), or `null` at the end of the file.

```
// Read a CSV file line by line      (inside a method that "throws IOException")
try (BufferedReader br = new BufferedReader(new FileReader("students.csv"))) {
    String line;
    while ((line = br.readLine()) != null) {
        String[] parts = line.split(",");
        System.out.println(parts[0] + " -> " + parts[1]);
    }
}

// Write lines efficiently
try (BufferedWriter bw = new BufferedWriter(new FileWriter("report.txt"))) {
    bw.write("Student report");
    bw.newLine();                    // platform-independent line break
    bw.write("Total: 42");
}
```

Example 4: Chaining Streams with Data Streams




```
try (DataOutputStream out = new DataOutputStream(
        new BufferedOutputStream(new FileOutputStream("scores.bin")))) {
    out.writeInt(3);
    out.writeDouble(17.5);
    out.writeUTF("Fahd");
}
try (DataInputStream in = new DataInputStream(
        new BufferedInputStream(new FileInputStream("scores.bin")))) {
    int n = in.readInt();
    double grade = in.readDouble();
    String name = in.readUTF();       // same order as written!
}
```

Each wrapper adds **one** feature to the stream it wraps: this is the **Decorator** design pattern. Closing the outer stream closes the whole chain.

Example 5: Saving Objects with Object Streams

```
class Student implements Serializable {
    private String name;
    private double grade;
    // constructor, getters, toString...
}

// Save an object (serialization)
try (ObjectOutputStream out = new ObjectOutputStream(
        new FileOutputStream("student.ser"))) {
    out.writeObject(new Student("Sara", 16.5));
}

// Load it back (deserialization)
try (ObjectInputStream in = new ObjectInputStream(
        new FileInputStream("student.ser"))) {
    Student s = (Student) in.readObject();
}
```

- **Serialization** turns a whole object into a sequence of bytes, so it can be saved to a file or sent over a network.
- The class must implement `Serializable`, a **marker interface**: it has no methods at all!
- `readObject()` returns an `Object`: cast it back to the right type.
- It can also throw `ClassNotFoundException`.
- Mark a field `transient` to exclude it (e.g. a password).

Reading the Keyboard and the Modern Files API

Bridge: bytes → characters

```
InputStreamReader isr =
        new InputStreamReader(System.in);
BufferedReader console = new BufferedReader(isr);
System.out.print("Your name: ");
String name = console.readLine();
```

- `System.in` is an `InputStream` (bytes) and `System.out` is a `PrintStream`.
- `InputStreamReader` converts a byte stream into a character stream.

Shortcut: java.nio.file.Files (JDK 11+)

```
Path path = Path.of("notes.txt");
Files.writeString(path, "Hello " + name);
List<String> lines = Files.readAllLines(path);
byte[] raw = Files.readAllBytes(path);
```

- One line to read or write a whole **small** file: Java opens and closes the streams for you.
- For large files, or data that arrives progressively, streams are still the right tool.

IO Streams Challenge

- Write a `GradeBook` program that:
  - reads `students.csv` line by line with a `BufferedReader`;
  - creates a `Student` object (name, grade) for each line;
  - computes the class average, the best and the worst student;
  - writes a `report.txt` file using a `BufferedWriter` (or a `PrintWriter`).
- Every stream must be closed with **try-with-resources**.
- If the input file is missing, print a friendly message instead of a stack trace.
- **Bonus:** save the list of students to `students.ser` with an `ObjectOutputStream`, then load it back.

students.csv

```
Sara,16.5
Omar,12.0
Lina,18.25
Youssef,9.75
```

report.txt

```
Students : 4
Average  : 14.13
Best     : Lina (18.25)
Worst    : Youssef (9.75)
```

Enumeration

- The enum type is Java's type to support something called an enumeration.
- Wikipedia defines enumeration as, “A complete ordered listing of all the items in a collection."
- Java describes the enum type as: A special data type that contains predefined constants.
- A constant is a variable whose value can't be changed, once its value has been set.
- An enum is a little like an array, except its elements are known, not changeable, and each element can be referred to by a constant name, instead of an index position.

```
public enum DayOfTheWeek {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
}
```

Enumeration

```
public enum DayOfTheWeek {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
}
```

- An enum in its simplest form, is described like a class. However, the keyword enum replaces the keyword class.
- You can name the enum with any valid identifier, but like a class, upper camel case is the preferred style.
- Within the enum body, you declare a list of constant identifiers, separated by commas. By convention, these are all uppercase labels.
- An enum is ordered, by the way you declare the constants.
- This means that SUNDAY is considered the first day of the week, and SATURDAY is the last day of the week, in this example.

Enumeration

```
public enum DayOfTheWeek {

    SUN, MON, TUES, WED, THURS, FRI, SAT
}
```

```
for (int i = 0; i < 10; i++ ) {
    weekDay = getRandomDay();

    System.out.printf("Name is %s, Ordinal Value = %d%n",
            weekDay.name(), weekDay.ordinal());

    if (weekDay == DayOfTheWeek.FRI) {
        System.out.println("Found a Friday!!!");
    }

    switchDayOfWeek(weekDay);
}
```

- Underneath the covers, the enum type is a special type of class, which contains fields to support the constants.
- You don't have to understand all the internals of an enum, to derive the benefits of using the type.
- Once you get used to how this type works, you will probably find many places to use an enum.
- They simplify your code and make it more readable in many ways.

Generalization and Abstraction

- We use the terms Abstraction and Generalization, when we start trying to model real world things in software.
- When you start modeling objects for your application, you start by identifying what features and behavior your objects have in common.
- We generalize when we create a class hierarchy.
- A base class is the most general class, the most basic building block, which everything can be said to have in common.

Generalization and Abstraction

- Part of generalizing is using abstraction.
- You can generalize a set of characteristics and behavior into an abstract type.
- If you consider an octopus, a dog, and a penguin, you will probably say they're all animals.
- An animal is really an abstract concept.
- An animal doesn't really exist, except as a way to describe a set of more specific things.
- If you can't draw it on a piece of paper, it's probably abstract.
- Abstraction simplifies the view of a set of items' traits and behavior, so we can talk about them as a group, as well as generalize their functionality.

Java’s support for Abstraction

- Java supports abstraction in several different ways.
  - Java allows us to create a class hierarchy, where the top of the hierarchy, the base class, is usually an abstract concept, whether it's an abstract class or not.
  - Java lets us create abstract classes.
  - Java gives us a way to create interfaces.

Abstract method

- An abstract method has a method signature, and a return type, but doesn't have a method body.
- Because of this, we say an abstract method is **unimplemented**.
- Its purpose is to describe behavior, which any object of that type will always have.
- Conceptually, we can understand behaviors like move or eat on an Animal, so we might include those as abstract methods, on an abstract type.
- You can think of an abstract method as a contract.
- This contract promises that all subtypes will provide the promised functionality, with the agreed upon name and arguments.

Concrete method

- A concrete method has a method body, usually with at least one statement.
- This means it has operational code, that gets executed, under the right conditions.
- A concrete method is said to implement an abstract method, if it overrides one.
- Abstract classes and interfaces can have a mix of abstract and concrete methods.

Method modifiers

- In addition to access modifiers, methods have other modifiers, which I'll list here, as a high-level introduction.

Modifier	Purpose
abstract	When you declare a method abstract, a method body is always omitted. An abstract method can only be declared on an abstract class or an interface.
static	Sometimes called a class method, rather than an instance method, because it's called directly on the Class instance.
final	A method that is final cannot be overridden by subclasses.
default	This modifier is only applicable to an interface, and I'll talk about later
native	This is another method with no body, but it's very different from the abstract modifier. The method body will be implemented in platform-dependent code, typically written in another programming language such as C. This is an advanced topic and not generally commonly used, and we won't be covering it in this course.
synchronized	This modifier manages how multiple threads will access the code in this method.

Abstraction – what is it ?

- Now consider these sentences.
  - We adopted a new pet this weekend.
  - I ordered something I really wanted from the store.
  - I bought a ticket and won a prize.
- If I said any of these things to a friend or coworker, it might be frustrating for them.
- I haven't given them enough information in each of these cases.
- They can't visualise what I am talking about, because they lack details. Which I didn't provide.
- New pet, something ordered, and a ticket, are too general when talking about one item.

Abstraction – what is it ?

- On the other hand, when we talk about groups of things, we don't usually need too many specifics.
- Consider these sentences.
  - I need to get home to feed the animals.
  - I'm waiting for my box of stuff from an online store to be delivered.
- So here, animals, and stuff are probably enough information, to fully describe the situation.

The abstract class

- The abstract class is declared with the abstract modifier.
- Here I declare an abstract class called Animal.

```
abstract class Animal {}     // An abstract class is declared with the abstract
                             // modifier.
```

- An abstract class is a class that's incomplete.
- You can't create an instance of an abstract class.

```
Animal a = new Animal();  // INVALID, an abstract class never gets instantiated
```

- An abstract class can still have a constructor, which will be called by its subclasses during their construction.

The abstract class

- An abstract class's purpose is to define the behavior its subclasses are required to have, so it always participates in **inheritance**.
- For the examples, assume that Animal is an abstract class.
- Classes extend abstract classes and can be concrete.
- Here, Dog extends Animal. Animal is abstract, but Dog is concrete.

```
class Dog extends Animal {}  // Animal is abstract, Dog is not
```

The abstract class

- A class that extends an abstract class can also be abstract itself.
- Mammal is declared abstract, and it extends Animal, which is also abstract.

```
abstract class Mammal extends Animal {}  // Animal is abstract, Mammal is also
                                         // abstract
```

- And finally an abstract class can extend a concrete class.
- Here we have BestOfBreed, an abstract class, extending Dog, which is concrete.

```
abstract class BestOfBreed extends Dog {}  // Dog is not abstract, but
                                           // BestOfBreed is
```

What is an abstract method ?

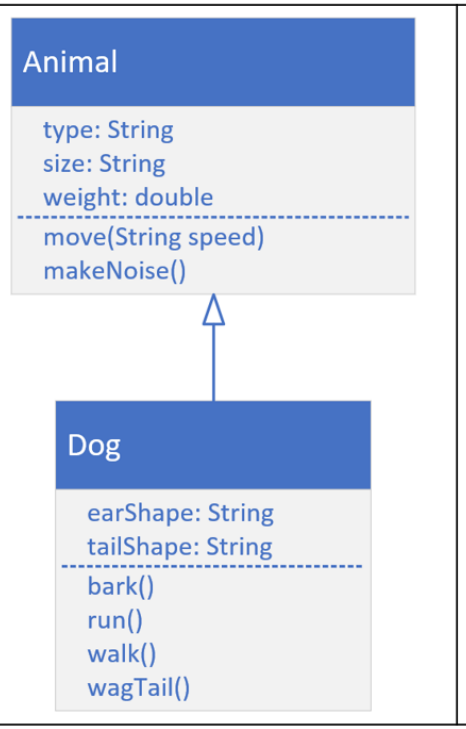
```
abstract class Animal {

    public abstract void move();
}
```

- An abstract method is declared with the modifier abstract.
  - It simply ends with a semi-colon.
  - It doesn't have a body, not even curly braces.
- An abstract method tells the outside world that all Animals will move, in the example I show here.
- Any code that uses a subtype of Animal, knows it can call the move method, and the subtype will implement this method with this signature.
- This is also true for a concrete class, and a concrete method that's overridden.
- You might be asking, what's the difference, and when would you use an abstract class.

What is an abstract method ?

- In the inheritance section, we created a very basic Animal class, and then we extended it to create a Dog.



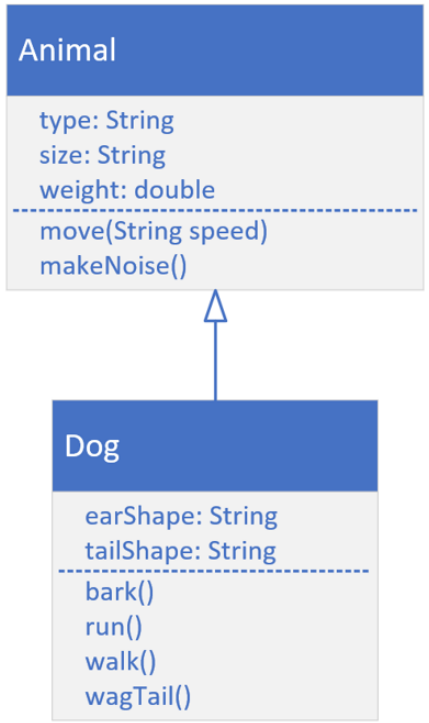
In that example, Animal was a concrete class, so the move and makeNoise methods had code in their method bodies.

Subclasses have choices when they extend a concrete class with concrete methods.

- They can inherit the same behavior from their parent. This means they don't have to even declare the methods in their class bodies.
- They can override the behavior from their parent. This means they have a method with the same signature, but with their own code in there, ignoring the parent's code altogether.
- They can also override the behavior but leverage the parent's method code, by calling the parent's method, using super in their overridden code.

What is an abstract method ?

- What happens if Animal is declared as abstract, and the move and makeNoise methods are also abstract?



If Animal is abstract and it's methods are abstract, subclasses no longer have the options I just talked about.

There is no concrete method for a subclass to inherit code from.

Instead, the subclass must provide a concrete method for any abstract method declared on its parent.

The subclass won't compile if it doesn't implement the abstract methods.

An Abstract class doesn't have to implement abstract methods

- An abstract class that extends another abstract class has some flexibility.
  - It can implement all of the parent's abstract methods.
  - It can implement some of them.
  - Or it can implement none of them.
  - It can also include additional abstract methods, which will force subclasses to implement both Animal's abstract methods, as well as Mammal's.

Why use an abstract class?

- In truth, you may never need to use an abstract class in your design, but there are some good arguments for using them.
- An abstract class in your hierarchy forces the designers of subclasses to think about, and create unique and targeted implementations, for the abstracted methods.
- It may not always make sense to provide a default, or inherited implementation of a particular method.
- An abstract class can't be instantiated, so if you're using abstract classes to design a framework for implementation, this is definitely an advantage.
- In our example, we don't really want people creating instances of Animals or Mammals.
  - We used those classes to abstract behavior at different classification levels.
  - All Animals have to implement the move and makeNoise methods, but only Mammals needed to implement shedHair, as I demonstrated.

Abstract Class Challenge

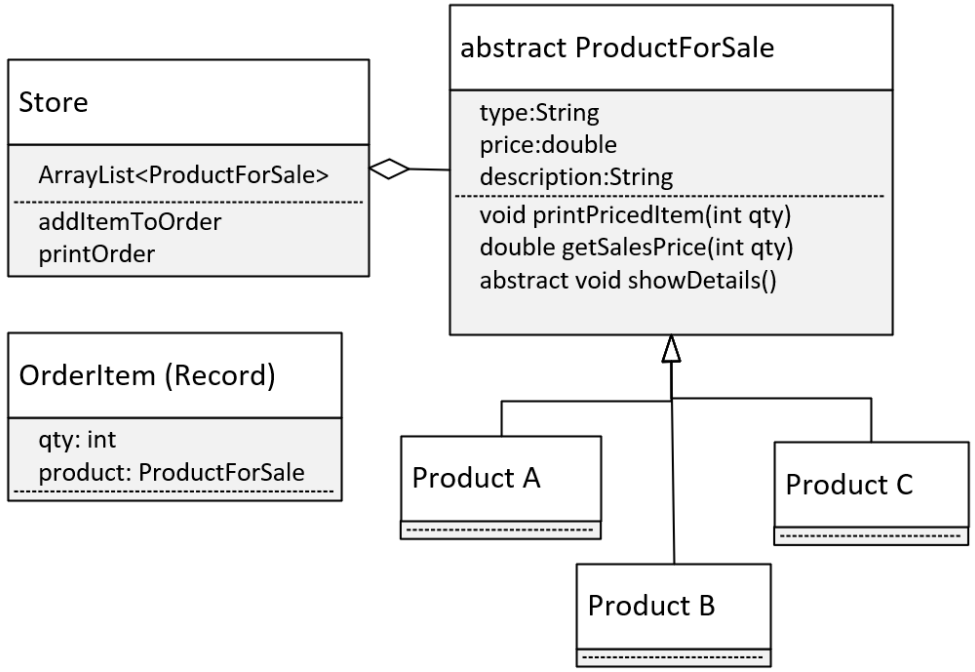
- In this challenge, you need to build an application that can be a store front for any imaginable item for sale.
- Instead of the Main class we usually create, create a Store class with a main method.
- The Store class should:
  - manage a list of products for sale, including displaying the product details.
  - manage an order, which can just be a list of OrderItem objects.
  - have methods to add an item to the order, and print the ordered items, so it looks like a sales receipt.

Abstract Class Challenge

- Also create a **ProductForSale** class that should have at least three fields: a **type, price,** and a **description**. This class should also have methods to:
  - getSalesPrice, a concrete method, which takes a quantity, and returns the quantity times the price.
  - printPricedItem, a concrete method, which takes a quantity, and should print an itemized line item for an order, with quantity and line-item price.
  - showDetails, an abstract method, which represents what might be displayed on a product page. Product type, description, price, and so on.
- Also create an **OrderItem** type, that has at a minimum 2 fields, **quantity** and a **Product for Sale**.
- You should also create **two or three classes that extend the ProductForSale class**, that will be products in your store.

Abstract Class Challenge: Design

- Here is the class diagram of what you'll be building.
- This covers all the requirements I talked about.
- Notice I'm specifying that OrderItem will be a record. This is just to keep the code simple.
- I'm not really specifying what our store products are, you can really put anything there.



Interface vs. Abstract Class

- You saw that an abstract class requires its subclasses to implement its abstract methods.
- An interface is similar to an abstract class, although it isn't a class at all.
- It's a special type, that's more like a contract between the class and client code, that the compiler enforces.
- By declaring it's using an interface, your class must implement all the abstract methods on the interface.
- A class agrees to this because it wants to be known by that type, by the outside world or client code.
- An interface lets classes that might have little else in common be recognized as a special reference type

Declaring an interface

- Declaring an interface is similar to declaring a class, using the keyword interface, where you would use the keyword class.
- Here, I'm declaring a public interface named FlightEnabled.

```
public interface FlightEnabled {}
```

- An interface is usually named according to the set of behaviors it describes.
- Many interfaces will end in 'able', like Comparable, and Iterable. Meaning something is capable or can perform a given set of behaviors.

Using an interface

- A class is associated to an interface by using the implements clause in the class declaration.
- Here, the class Bird implements the FlightEnabled interface.

```
public class Bird implements FlightEnabled {

}
```

- Because of this declaration, we can use FlightEnabled as the reference type and assign it an instance of bird.
- In this code sample, I create a new Bird object but assign it to the FlightEnabled variable named flier.

```
FlightEnabled flier = new Bird();
```

Using extends and implements in same declaration

- A class can only extend a single class, which is why Java supports only single inheritance.
- However, a class can implement many interfaces, providing flexibility and modularity. This allows for the combination of different sets of behaviors, making interfaces a powerful feature.
- A class can both extend another class and implement one or more interfaces.

```
public class Bird extends Animal implements FlightEnabled, Trackable {
```

  - In this example, the Bird class extends or inherits from Animal, but it's implementing both a FlightEnabled, and Trackable interface.
  - We can describe Bird by what it is and what it does.

The abstract modifier is implied on an interface

- I don't have to declare the interface type abstract, because this modifier is implicitly declared for all interfaces.

```
abstract interface FlightEnabled {  // abstract modifier here is unnecessary
                                    // and redundant
```

- Likewise, I don't have to declare any method abstract.
- In fact, any method declared without a body, is really **implicitly declared both public and abstract.**
- The three declarations shown on this slide, result in the same thing, under the covers:

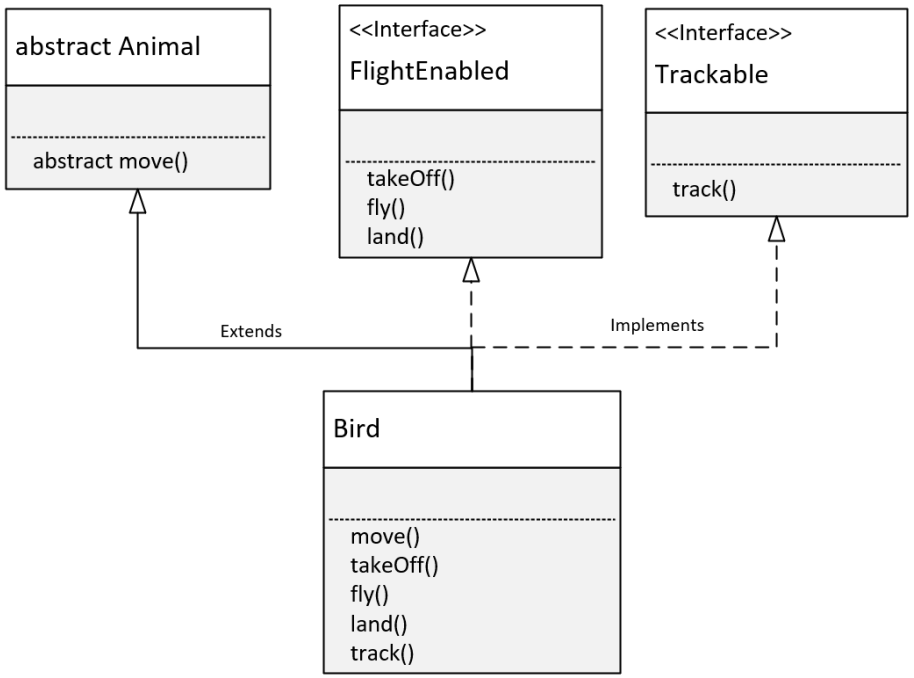
```
public abstract void fly();     // public and abstract modifiers are redundant,
                                // meaning unnecessary to declare
abstract void fly();            // abstract modifier is redundant, meaning
                                // unnecessary to declare
void fly();                     // This is PREFERRED declaration, public and
                                // abstract are implied.
```

All members on an interface are implicitly public

- If you omit an access modifier on a class member, it's implicitly package private.
- If you omit an access modifier on an interface member, it's **implicitly public**.
- This is an important difference, and one you need to remember.
- Changing the access modifier of a method to **protected** on an interface, is **a compiler error**, whether the method is concrete or abstract.
- Only a **concrete method** can have **private** access.

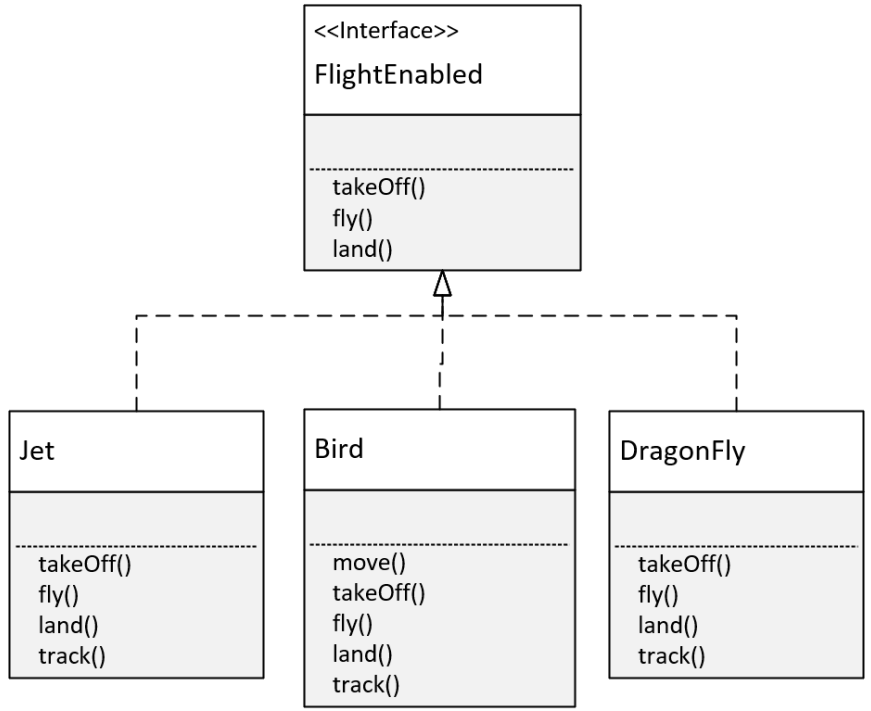
The Bird Class

- An interface lets us treat an instance of a single class as many different types.
- The Bird Class inherits behavior and attributes from Animal, because I used the extends keyword in the declaration of Bird.
- Because the move method was abstract on Animal, Bird was required to implement it.
- The Bird Class implements the FlightEnabled interface.
- This required the Bird class to implement the takeOff, fly, and land methods which were the abstract methods on FlightEnabled.
- The Bird Class also implements the Trackable interface.
- This required the Bird class to implement the track method, which was the abstract method declared on Trackable.



The FlightEnabled Interface

- Interfaces let us take objects that may have almost nothing in common, and write reusable code so we can process them all in a similar manner.
- On this slide, you can see that a Jet, a Bird, and a DragonFly are very different entities.
- But because they implement FlightEnabled, we can treat them all as the same type, as something that flies, and ignore the differences in the classes.
- Interfaces allow us to type our objects differently, **by behavior only**.



The final modifier

- When we use the final modifier, we prevent any further modifications to that component.
  - a final method means it can't be overridden by a subclass.
  - a final field means an object's field can't be reassigned or given a different value after its initialization.
  - a final static field is a class field that can't be reassigned or given a different value after the class's initialization process.
  - a final class can't be overridden, meaning no class can use it in the extends clause.
  - a final variable in a block of code, means that once it's assigned a value any remaining code in the block can't change it.
  - a final method parameter means we can't assign a different value to that parameter in the method code block.

Constants in Java

- A constant in Java is a variable that can't be changed.
- A constant variable is a final variable of primitive type, or type String, that is initialized with a constant expression.
- Constants in Java are usually named with all uppercase letters and with underscores between words.
- A static constant means we access it via the type name.
- We saw this with the INTEGER.MAX_VALUE, and the INTEGER.MIN_VALUE fields.

A field declared on an Interface is always public, static and final

- Java lets us specify these like an ordinary field on an interface, which might be kind of confusing and misleading to a new Java programmer.
- But we can declare them with any combination of those modifiers, or none at all with the same result.
- These all mean the same thing on an interface.

```
double MILES_TO_KM = 1.60934;
final double MILES_TO_KM = 1.60934;
public final double MILES_TO_KM= 1.60934;
public static final double MILES_TO_KM = 1.60934;
```

What's happened to the Interface since JDK 8
the default method (as of JDK8)

- Before JDK 8, the interface type could only have public abstract methods.
- JDK 8 introduced the default and public static methods and JDK 9 introduced private methods, both static and non-static.
- All of these new method types (on an interface) are concrete methods.
- An extension method is identified by the modifier default, so it's more commonly known as the default method.
- This method is a concrete method, meaning it has a code block and we can add statements to it.
- In fact, it has to have a method body, even if the body is just an empty set of curly braces.
- It's a lot like a method on a superclass because we can override it.
- Adding a default method doesn't break any classes currently implementing the interface.

What's happened to the Interface since JDK 8
the default method (as of JDK8)

- Just like overriding a method on a class, you have three choices when you override a default method on an interface.
  - You can choose not to override it at all.
  - You can override the method and write code for it, so that the interface method isn't executed.
  - Or you can write your own code, and invoke the method on the interface, as part of your implementation.

public static methods on an interface (as of JDK8)

- Another enhancement that Java included in JDK 8, was support for public static methods on an interface.
- Static methods don't need to specify a public access modifier, because it's implied.
- When you call a public static method on an interface, you must use the interface name as a qualifier.
- In the ArrayList lectures, you may remember I used two static helper methods, on the Comparator interface which were added in JDK 8.
- These were Comparator.naturalOrder() and Comparator.reverseOrder().

Private methods (JDK 9)

- JDK 9 gave us private methods, both static and not static.
- This enhancement primarily addresses the problem of re-use of code within concrete methods on an interface.
- A private static method can be accessed by either a public static method, a default method, or a private non-static method.
- A private non-static method is used to support default methods and other private methods.

Method Type	Modifier	Allowed Since	Callable From	Purpose
public abstract	implicit	Java 1.0	Implementing class	Contract definition
default	explicit	Java 8	Subclasses / implementing classes	Provide default implementation
static	explicit	Java 8	Interface only	Utility methods
private static	explicit	Java 9	Interface (internal use only)	Reuse helper code for static methods
private (non-static)	explicit	Java 9	Interface (internal use only)	Reuse helper code for default methods

Interface vs. Abstarct class

- I've said that interfaces and abstract classes are both abstracted types, and abstracted types are used as reference types in code.
- The table on this slide is a summary of the similarities and differences.

	Abstract Class	**Interface**
An instance can be created from it	No	No
Has a constructor	Yes	No
Implemented as part of the Class Hierarchy. Uses Inheritance	Yes (in extends clause)	No (in implements clause)
records and enums can extend or implement?	No	Yes
Inherits from java.lang.Object	Yes	No
Can have both abstract methods and concrete methods	Yes	Yes (as of JDK 8)
Abstract methods must include abstract modifier	Yes	No (Implicit)
Supports default modifier for it's methods	No	Yes (as of JDK 8)
Can have instance fields (non-static instance fields)	Yes	No
Can have static fields (class fields)	Yes	Yes - (implicitly **`public static final`**)

The Interface Challenge

- In this challenge, you'll be working on creating some mappable output.
- In the past decade or so, maps have become part of so many applications.
- Most things, when drawn on a map, fall into three categories, a point, a line, or a polygon or geometric shape.
- The result of your code will be text that could be printed out to a file for exchanging data with a mapping application.
- One such file is a specially formatted file, called geojson, which is a JSON file extended for geographical elements.
- You don't have to know JSON or geojson to be successful at this challenge.

The Interface Challenge

- In this challenge, you'll be working on creating some mappable output.
- In the past decade or so, maps have become part of so many applications.
- Most things, when drawn on a map, fall into three categories, a point, a line, or a polygon or geometric shape.
- The result of your code will be text that could be printed out to a file for exchanging data with a mapping application.
- One such file is a specially formatted file, called geojson, which is a JSON file extended for geographical elements.
- You don't have to know JSON or geojson to be successful at this challenge.
- For this challenge, you'll simply create a String for every feature that will be mapped.
- An example of such a String is shown here:

```
"properties": { "name": "Sydney Opera House", "usage": "Entertainment"}
```

The Interface Challenge

- First, create a **Mappable** Interface.
- The interface should **force classes to implement three methods**.
  - One method should return a **label** (how the item will be described on the map).
  - One should return a **geometry type** (POINT or LINE) which is what the type will look like on the map.
  - The last should return an icon **type** (sometimes called a map marker).

The Interface Challenge

- In addition to the three methods described, the interface should also include:
  - A constant String value called JSON_PROPERTY, which is equal to: "properties":{%s}. A hint here, using a text block will help maintain quotation marks in your output.
  - Include a default method called toJSON() that prints out the type, label, and marker. I'll show examples shortly.
  - A **static method**, that takes a Mappable instance as an argument. This method should print out the properties for each mappable type, including those mentioned above, but also any other fields on the business classes.

The Interface Challenge

- You'll also want to create two classes that implement this interface, a Building and UtilityLine.
  - One class, in my case the Building, should have a geometry type of POINT, and one class should have a geometry type of Line. The UtilityLine class will be my example for a class that will be a LINE on a map.
  - The Building will be shown on a city map, as a point with the icon and label specified and the Utility Line will be a line on the map.

The Interface Challenge

- Your final output should look something like I show on this slide.
- You should output the geometry type, the icon information, and the label.
- Here is an example for a building, including type, label, and marker, but also the building name and usage, which are fields on Building.


```
"properties": {"type": "POINT", "label": "Sydney Town Hall (GOVERNMENT)", "marker": "RED STAR", "name": "Sydney Town Hall", "usage": "GOVERNMENT"}
```

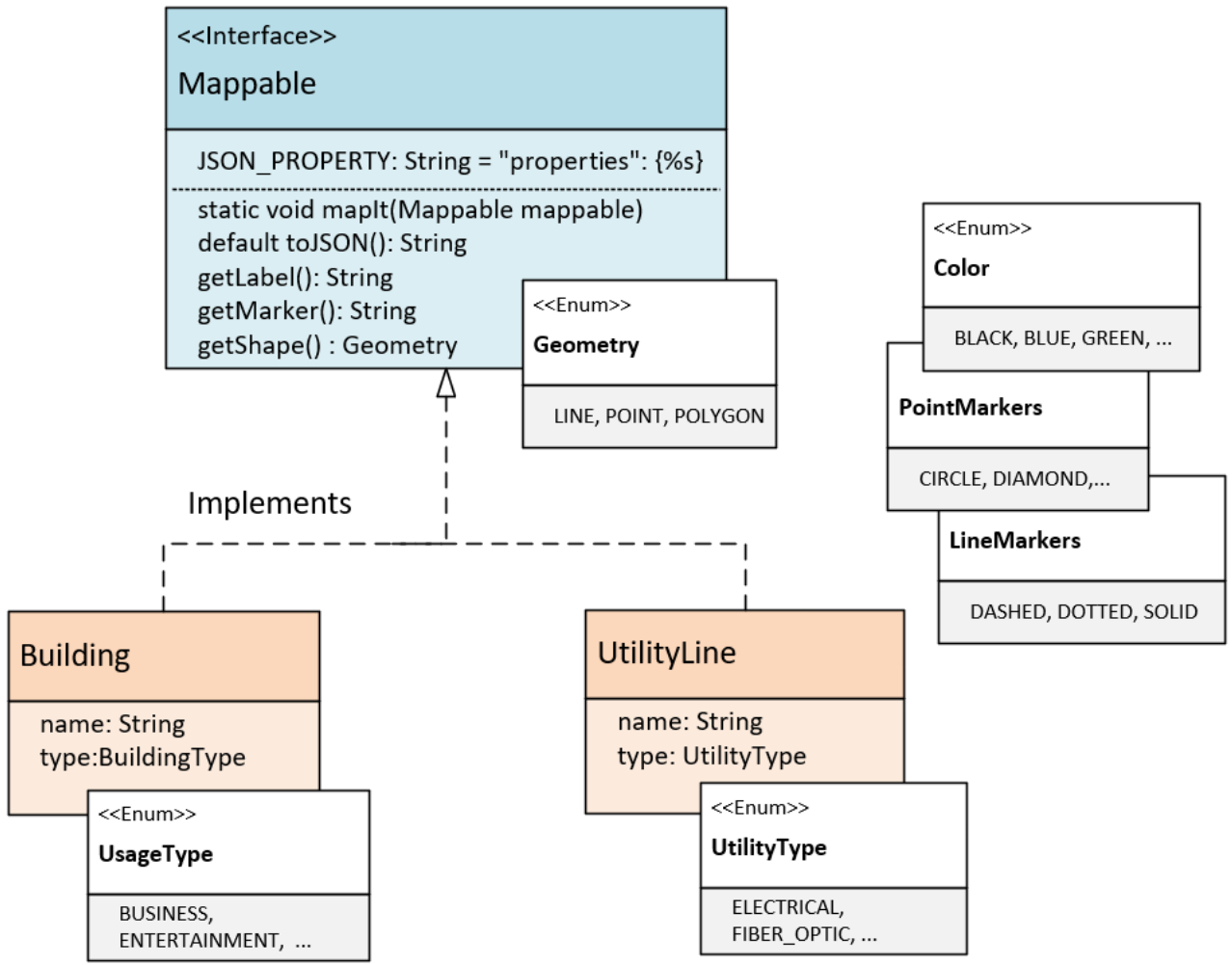

- And here is an example for a fiber optic Utility line, so a LINE, a green dotted line, would get drawn for a fiber optic cable on College Street.


```
"properties": {"type": "LINE", "label": "College St (FIBER_OPTIC)", "marker": "GREEN DOTTED", "name": "College St", "utility": "FIBER_OPTIC"}
```


- You can see that the properties are a comma delimited list in curly braces with the property or field name in quotes, then a colon, followed by the property value or field value, and that's also in double quotes.
- You can see that the properties are a comma delimited list in curly braces with the property or field name in quotes, then a colon, followed by the property value or field value, and that's also in double quotes.

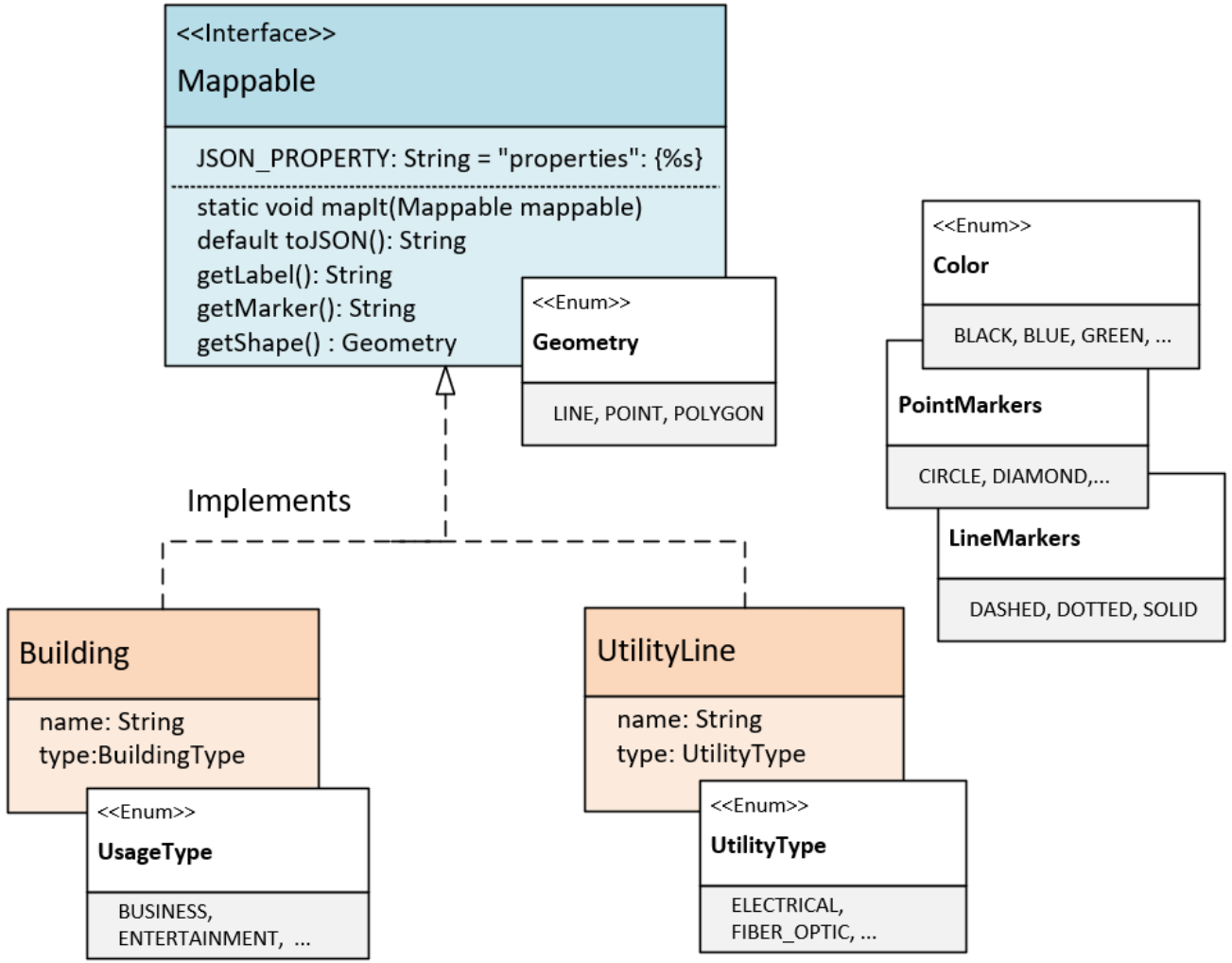
The Class Diagram

- On the Mappable interface, I have one constant, JSON_PROPERTY, and 3 abstract methods, getLabel, getMarker, and getShape.
- I'm also including a default method, toJSON, which is going to return a String.
- The getShape method returns an enum type, Geometry, and the valid types on this enum are LINE, POINT, and POLYGON.
- Use enums for Color, and the PointMarker and LineMarker types.



The Class Diagram

- Two business classes to be mapped, Building and UtilityLine.
- For Building, Use an enum to describe the building type.
- For the UtilityLine, use enum to describe the type of utility it is.



Questions ?

